Reviewer Pack — Minimal Ehrhart Semialgebra Rank and Communication Complexit…

Entry e07ae4e22c49 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 07:30:29 UTC

Minimal Ehrhart Semialgebra Rank and Communication Complexity Rank Inequality

Entry ID: `e07ae4e22c49` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 07:30:29 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Ehrhart Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every instance of an n-communication protocol, the rank of its associated semialgebra in the Ehrhart theory is upper-bounded by a function O(n log n), which implies that communication complexity rank is O(n log n).

Rationale (proposer's reasoning):

> Ehrhart theory studies the geometry of integer points in polytopes and has not been extensively applied to study communication complexity. This conjecture suggests that Ehrhart theory could provide new insights into the structure of communication complexity, potentially leading to better lower bounds.

Taxonomy category: `Ehrhart_theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `09cdbbfffa9675bb`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For every n-communication protocol, if the average semialgebra rank across all seeds is less than or equal to O(n log n)

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Ehrhart Theory" AND "communication complexity rank"` - `"semialgebra rank" AND "Communication Complexity"` - `"Minimal Ehrhart Semialgebra Rank" AND "Matrix Rank"`

Top relevant hits considered: - [s2:2407.01802] An XOR Lemma for Deterministic Communication Complexity - [s2:2310.03355] A Note on the LogRank Conjecture in Communication Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.3s

5.1 Generated Python source

```python
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_rank(matrix):
    n = len(matrix)
    rank = 0
    for i in range(n):
        if all(matrix[j][i] == 0 for j in range(rank)):
            continue
        pivot_row = rank
        for j in range(pivot_row + 1, n):
            if matrix[j][i] != 0:
                matrix[pivot_row], matrix[j] = matrix[j], matrix[pivot_row]
                break
        else:
            continue
        rank += 1
        denom = matrix[pivot_row][i]
        for j in range(n):
            if j == i:
                matrix[pivot_row][j] = Fraction(1, denom)
            else:
                matrix[pivot_row][j] = -Fraction(matrix[pivot_row][j], denom)
        for j in range(n):
            if j != pivot_row:
```

```
                factor = matrix[j][i]
                for k in range(n):
                    matrix[j][k] += -factor * matrix[pivot_row][k]
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_max = 40
    instances_tested = 30
    total_metric_value = 0.0
    conjecture_holds = True
    counterexample = ""

    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(instances_tested // len([5, 10, 15, 20, 30, 40])):
            # Generate a random communication protocol (example: a binary matrix)
            protocol = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

            # Convert the protocol to a semialgebraic matrix
            semialgebra_matrix = []
            for i in range(n):
                row = [protocol[i][j] * protocol[j][i] for j in range(n)]
                semialgebra_matrix.append(row)

            # Compute the rank of the semialgebraic matrix
            rank = matrix_rank(semialgebra_matrix)

            # Measure the communication complexity rank (example: number of 1s in the protocol)
            comm_complexity_rank = sum(sum(row) for row in protocol)

            # Check if the inequality holds
            if rank > n * math.log(n, 2):
                conjecture_holds = False
                counterexample = f"n={n}, semialgebra_rank={rank}, comm_complexity_rank={comm_comp
                break

            total_metric_value += rank

    return {
        "metric_name": "semialgebra_rank",
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested * len([5, 10, 15, 20, 30, 40]),
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
```

```
    total_metric_values = [r["metric_value"] for r in results]
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(total_metric_values)/len(total_metric_values):.6f} std
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(total_metric_values)/len(total_metric_values):.6f} std
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
tric_name": "semialgebra_rank", "metric_value": 0.000000, "instances_tested": 180, "n_max": 40, "con
TRIAL: {"seed": 547, "metric_name": "semialgebra_rank", "metric_value": 0.000000, "instances_tested"
TRIAL: {"seed": 593, "metric_name": "semialgebra_rank", "metric_value": 0.000000, "instances_tested"
TRIAL: {"seed": 631, "metric_name": "semialgebra_rank", "metric_value": 0.000000, "instances_tested"
TRIAL: {"seed": 677, "metric_name": "semialgebra_rank", "metric_value": 0.000000, "instances_tested"
TRIAL: {"seed": 727, "metric_name": "semialgebra_rank", "metric_value": 0.000000, "instances_tested"
TRIAL: {"seed": 773, "metric_name": "semialgebra_rank", "metric_value": 0.000000, "instances_tested"
TRIAL: {"seed": 821, "metric_name": "semialgebra_rank", "metric_value": 0.000000, "instances_tested"
TRIAL: {"seed": 877, "metric_name": "semialgebra_rank", "metric_value": 0.000000, "instances_tested"
TRIAL: {"seed": 929, "metric_name": "semialgebra_rank", "metric_value": 0.000000, "instances_tested"

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_da357522.py", line 105, in <module>
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)
                       ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^~~~~~~~~~~~~~
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means that the results could not be analyzed to confirm or falsify the conjecture. | next: Investigate and fix the crash in the test code to ensure it can complete without interruption.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22165
2	propose	ollama_remote	glm4:latest	0	0	12250
3	propose	ollama_remote	glm4:latest	0	0	12269
4	propose	ollama_remote	glm4:latest	0	0	13074
5	preregistration	ollama_remote	glm4:latest	0	0	9099
6	novelty	ollama_remote	glm4:latest	0	0	8081
7	novelty	ollama_remote	glm4:latest	0	0	9623
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13756
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12756
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13594
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13708
12	judge	ollama_remote	glm4:latest	0	0	17574

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 157949 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/e07ae4e22c49.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e07ae4e22c49.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e07ae4e22c49.tar.gz` (if generated)